Foundational Verification of Probabilistic Programs

Mechanising the Lilac probabilistic separation logic in Lean

Edwin Fernando

June 19, 2026

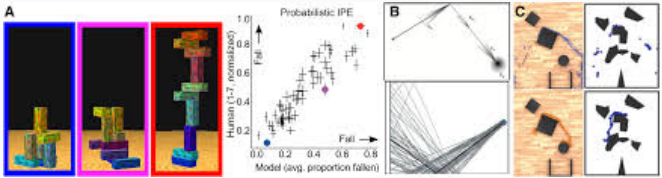
Department of Computing, Imperial College London

Probabilistic Programming is Everywhere

Probabilistic programming is everywhere

The most general feature of a PPL — sampling from a distribution — lives in the standard library of every major language.

- Cryptography — protocols, ZK proofs
- Differential privacy
- Randomised algorithms
- Distributed systems
- Bayesian inference & probabilistic ML
- Scientific simulation — climate, physics
- Robotics & perception — intuitive physics



Discrete vs. continuous

The dividing line that matters runs through the applications.

Discrete

Coin flips, Bernoulli / categorical choices, dice.

Cryptography, differential privacy, randomised algorithms — Applications traditionally within CS - much larger focus.

Continuous

Gaussian noise, sensor models, Bayesian priors over $\mathbb{R}$, physical quantities.

Scientific simulation, Bayesian inference, ML — where the real-world modelling happens.

Continuous distributions are where prior verification work fell short — and where this project makes its **first mechanised** contribution.

Verifying probabilistic programs is notoriously hard

- Conventional testing fails: a rare *correct* outlier execution and a genuine bug look *identical*.
- Correctness is a property of the whole distribution, not of any single run.
- Reasoning about independence and conditioning is subtle and error-prone by hand.

Verifying probabilistic programs is notoriously hard

- Conventional testing fails: a rare *correct* outlier execution and a genuine bug look *identical*.
- Correctness is a property of the whole distribution, not of any single run.
- Reasoning about independence and conditioning is subtle and error-prone by hand.

So we need proof

Machine-checked, **foundational** arguments about distributions — resting on nothing but the axioms of mathematics.

Contribution

The first mechanisation of foundational verification

for probabilistic programs

supporting continuous distributions.

Contribution

1. Soundness proofs
2. Interactive *Proof Mode*

using Iris

The first mechanisation of foundational verification

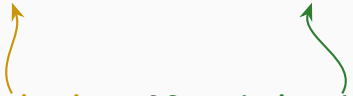
for probabilistic programs

supporting continuous distributions.

Contribution

1. Soundness proofs
2. Interactive *Proof Mode* using Iris

Using a theorem prover (Lean/Rocq/Agda) Assuming only axioms of math[1]



The first mechanisation of foundational verification

for probabilistic programs

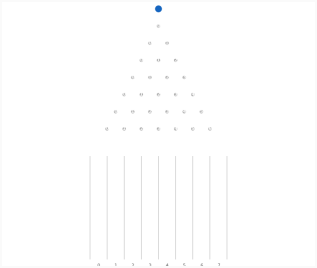
supporting continuous distributions.

[1] Background Sec 2.4.1 “Dependent Type Theory”

A program is a distribution over runs

$$\text{flip} : (m : \mathbb{R}) \to \text{Distrib}\ \{m, m+1\}$$

an individual "run" of the program



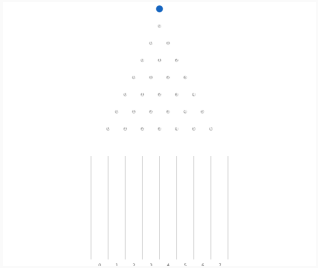
Honing in on a single marble

$$X_0 \leftarrow \text{flip } 0$$
$$X_1 \leftarrow \text{flip } X_0$$
$$X_2 \leftarrow \text{flip } X_1$$
$$\text{ret } X_2$$

$$|||$$

$$\text{for } (3,\ \text{flip } 0,\ \lambda X.\ \text{flip } X)$$

The program as a whole



All the marbles at once

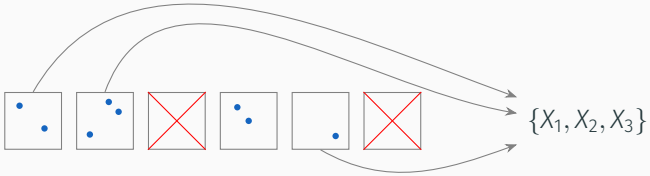
Probabilistic semantics

$$
\begin{aligned}
&X_0 \leftarrow \mathsf{flip}\ 0\\
&X_1 \leftarrow \mathsf{flip}\ X_0\\
&X_2 \leftarrow \mathsf{flip}\ X_1\\
&\quad \mathsf{ret}\ X_2
\end{aligned}
\qquad \equiv \qquad M : \qquad [\![\Gamma]\!] \to \mathrm{Measure}\ ty
$$

Refining the domain: the Hilbert cube

$$
\begin{aligned}
&X_0 \leftarrow \text{flip } 0 \\
&X_1 \leftarrow \text{flip } X_0 \\
&X_2 \leftarrow \text{flip } X_1 \\
&\quad \text{ret } X_2
\end{aligned}
\quad \equiv \quad
M : \mathrm{HC} \to [\![\Gamma]\!] \to \mathrm{Measure}\ ty
$$

HC = Hilbert cube $= [0,1]^{\mathbb{N}}$



How to verify: probabilistic separation logic

$$\underbrace{X \leftarrow \mathrm{unif}[0,1];\ \ \mathrm{ret}\ X}_{\text{program}}$$

$$\equiv$$

$$\underbrace{\lambda E \mapsto \int (\lambda \mu \mapsto \mu\, E)\ d\big(\mathrm{map}\ (\lambda X \mapsto \mathrm{dirac}\, X)\ \mathrm{unif}[0,1]\big)}_{\text{denotation}}$$

How to verify: probabilistic separation logic

$$\underbrace{X \leftarrow \mathrm{unif}[0,1];\quad \mathrm{ret}\ X}_{\text{program}}$$

$$\equiv$$

$$\underbrace{\lambda E \mapsto \int (\lambda \mu \mapsto \mu\, E)\, d(\mathrm{map}\ (\lambda X \mapsto \mathrm{dirac}\ X)\ \mathrm{unif}[0,1])}_{\text{denotation}}$$

How to verify: probabilistic separation logic

$$\underbrace{X \leftarrow \mathrm{unif}[0,1];\ \ \mathrm{ret}\ X}_{\text{program}}$$

$$\equiv$$

$$\underbrace{\lambda E \mapsto \int (\lambda \mu \mapsto \mu\, E)\, d(\mathrm{map}\,(\lambda X \mapsto \mathrm{dirac}\, X)\ \mathrm{unif}[0,1])}_{\text{denotation}}$$

reason directly about the variables

$$X \sim \mathrm{unif}[0,1]$$

Iris, and the logic of bunched implications (BI)

The separation logic is built on the Lean port of Iris; its assertions Prop form a BI (bunched-implication) logic.

$\ulcorner \cdot \urcorner$	$: \mathrm{Prop} \to \mathrm{Prop}$	pure embedding
$\mathsf{True},\ \mathsf{False},\ \mathsf{emp}$	$: \mathrm{Prop}$	units
$\wedge,\ \vee,\ \Rightarrow,\ *,\ -\!\!*$	$: \mathrm{Prop} \times \mathrm{Prop} \to \mathrm{Prop}$	binary connectives
$\forall,\ \exists$	$: \forall A.\ (A \to \mathrm{Prop}) \to \mathrm{Prop}$	quantifiers

$*$ separating conjunction $-\!\!*$ magic wand — the separation-logic-specific connectives.

Iris, and the logic of bunched implications (BI)

Assertions Prop form a **BI (bunched-implication) logic** on the Lean port of Iris.

$\ulcorner \cdot \urcorner$	$: \mathrm{Prop} \to \mathrm{Prop}$	pure embedding
$\mathsf{True}, \mathsf{False}, \mathsf{emp}$	$: \mathrm{Prop}$	units
$\wedge, \vee, \Rightarrow, *, -\!\!*$	$: \mathrm{Prop} \times \mathrm{Prop} \to \mathrm{Prop}$	binary connectives
$\forall, \exists$	$: \forall A.\ (A \to \mathrm{Prop}) \to \mathrm{Prop}$	quantifiers

Iris, and the logic of bunched implications (BI)

Assertions Prop form a **BI (bunched-implication) logic** on the Lean port of Iris.

$\ulcorner \cdot \urcorner$	$: \mathrm{Prop} \to \mathrm{Prop}$	pure embedding
$\mathsf{True}, \mathsf{False}, \mathsf{emp}$	$: \mathrm{Prop}$	units
$\wedge, \vee, \Rightarrow, *, -\!\!*$	$: \mathrm{Prop} \times \mathrm{Prop} \to \mathrm{Prop}$	binary connectives
$\forall, \exists$	$: \forall A.\ (A \to \mathrm{Prop}) \to \mathrm{Prop}$	quantifiers

- A logic inside a logic. Prop is an object logic inside Lean's meta-logic.[2]

Iris, and the logic of bunched implications (BI)

Assertions Prop form a **BI (bunched-implication) logic** on the Lean port of Iris.

$\ulcorner \cdot \urcorner$	$: \mathrm{Prop} \to \mathrm{Prop}$	pure embedding
$\mathsf{True}, \mathsf{False}, \mathsf{emp}$	$: \mathrm{Prop}$	units
$\wedge, \vee, \Rightarrow, *, -\!\!*$	$: \mathrm{Prop} \times \mathrm{Prop} \to \mathrm{Prop}$	binary connectives
$\forall, \exists$	$: \forall A.\ (A \to \mathrm{Prop}) \to \mathrm{Prop}$	quantifiers

- A logic inside a logic. Prop is an object logic inside Lean's meta-logic.[2]
- Impredicativity. $\forall/\exists$ range over an *arbitrary* type A — including Prop itself: we may quantify over propositions.

[2] Revisited later in the talk.

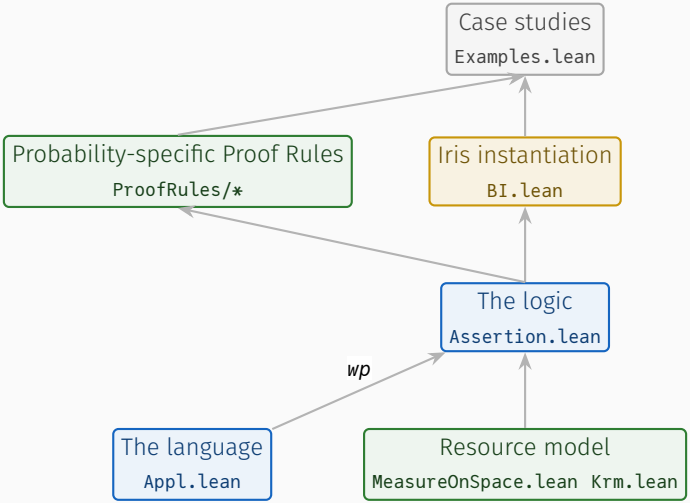
Mechanisation of Core[3] Lilac in Lean

a **continuous** probabilistic separation logic with separation as independence.

Along the way, plausibly the first (or second) instantiation of **Iris-lean** for a program logic of this kind.

[3] Extended Lilac adds reasoning about conditioning.

The formalisation, in one picture



wp_meas

- Uniform: suppose (1) $\gamma, D, \mathcal{P} \vdash \forall_{\forall\alpha}\Sigma\mathcal{A}.\ X \rightsquigarrow \mathrm{Unif}[0,1] \rightarrow Q$. To show $\mathsf{wp}(\mathrm{Unif}\ [0,1], X; \mathcal{A}, Q)$, suppose $\mathcal{P}_{\mathrm{frame}} \bullet \mathcal{P} \sqsubseteq (\Sigma_\Omega, \mu)$ and $\mathrm{Dist} : \mathsf{RV}[\![\Lambda_{\mathrm{dist}}]\!]$. Let n witness $(D_{\mathrm{dist}}, \mathcal{P}_{\mathrm{frame}} \bullet \mathcal{P})$'s finite footprint. Write the Hilbert cube as $[0,1]^{\mathbb{N}} \cong [0,1]^n \otimes [0,1]^{\mathbb{N}}$. Define μ' via this isomorphism as the product measure $\mu|_{\Omega_n}{}^{\infty} \otimes \lambda$, where λ assigns to each finite-dimensional box $\prod_{i=1}^k [a_i, b_i] \times [0,1]^{\mathbb{N}}$ the measure $\prod_{i=1}^k |b_i - a_i|$ and extends to a measure on the whole Hilbert cube by the Carathéodory extension theorem. Let $\mathcal{P}_\omega$ be the restriction of μ' to measurable sets of the form $[0,1]^n \times F \times [0,1]^{\mathbb{N}}$. Let X be the projection $\pi_{n+1} = (\ldots, \omega_{n+1}, \ldots) \mapsto \omega_{n+1}$. By construction, the composite $\mathcal{P}_{\mathrm{frame}} \bullet \mathcal{P} \bullet \mathcal{P}_\omega$ is defined and $\mathcal{P}_{\mathrm{frame}} \bullet \mathcal{P} \bullet \mathcal{P}_\omega \sqsubseteq (\Sigma_\Omega, \mu')$ and X is $\mathcal{P}_\omega$-measurable and uniformly distributed in $[0,1]$. Therefore $\gamma, (D, X), \mathcal{P} \star \mathcal{P}_\omega \vDash \Theta$ by (1), and it only remains to show

$$\left(\begin{array}{l} \omega \leftarrow \mu; \\ v \leftarrow \mathrm{Unif}\ [0,1]; \\ \mathrm{ret}\ (D_{\mathrm{ext}}(\omega), D(\omega), v) \end{array}\right) = \left(\begin{array}{l} \omega \leftarrow \mu'; \\ \mathrm{ret}\ (D_{\mathrm{ext}}(\omega), D(\omega), X(\omega)) \end{array}\right)$$

$$\left(\begin{array}{l} \omega \leftarrow \mu; \\ v \leftarrow \mathrm{Unif}\ [0,1]; \\ \mathrm{ret}\ (D_{\mathrm{ext}}(\omega), D(\omega), v) \end{array}\right) = \left(\begin{array}{l} \omega_{1\cdots n} \leftarrow \mu|_{\{1\cdots n\}}; \\ v \leftarrow \mathrm{Unif}\ [0,1]; \\ \mathrm{ret}\ (D_{\mathrm{ext}}(\omega_{1\cdots n}), D(\omega_{1\cdots n}), v) \end{array}\right)$$

$$= \left(\begin{array}{l} (\omega_{1\cdots n}, v) \leftarrow \mu|_{\{1\cdots n\}} \otimes \mathrm{Unif}\ [0,1]; \\ \mathrm{ret}\ (D_{\mathrm{ext}}(\omega_{1\cdots n}), D(\omega_{1\cdots n}), v) \end{array}\right) = \left(\begin{array}{l} \omega_{1\cdots n+1} \leftarrow \mu'|_{\{1\cdots n+1\}}; \\ \mathrm{ret}\ (D_{\mathrm{ext}}(\omega_{1\cdots n}), D(\omega_{1\cdots n}), \omega_{n+1}) \end{array}\right)$$

fp wit meas pcm bind post

wp_meas

- Uniform: suppose (1) $\gamma, D, \mathcal{P} \vdash \forall_{\mathbb{N}} X{:}\mathcal{A}.\ X \rightsquigarrow \mathrm{Unif}[0,1] \rightarrow Q$. To show $\mathrm{wp}(\mathrm{Unif}\ [0,1], X{:}\mathcal{A}, Q)$, suppose $\mathcal{P}_{\mathrm{frame}} \bullet \mathcal{P} \sqsubseteq (\Sigma_{\mathrm{cl}}, \mu)$ and $\mathrm{Dest} : \mathbb{N}\,\overline{\mathbb{V}}[\![\mathrm{A}_{\mathrm{est}}]\!]$. Let n witness $(D_{\mathrm{est}}, \mathcal{P}_{\mathrm{frame}} \bullet \mathcal{P})$'s finite footprint. Write the Hilbert cube as $[0,1]^{\mathbb{N}} \cong [0,1]^n \otimes [0,1]^{\mathbb{N}}$. Define μ' via this isomorphism as the product measure $\mu|_{\{0,1\}^n} \otimes \lambda$, where λ assigns to each finite-dimensional box $\prod_{i=1}^{n} [a_i, b_i] \times [0,1]^{\mathbb{N}}$ the measure $\prod_{i=1}^{n} |b_i - a_i|$ and extends to a measure on the whole Hilbert cube by the Carathéodory extension theorem. Let $\mathcal{P}_n$ be the restriction of μ' to measurable sets of the form $[0,1]^n \times F \times [0,1]^{\mathbb{N}}$. Let X be the projection $x_{n+1} = (\ldots, \omega_{0:n}, \ldots) \mapsto \omega_{0:n}$.
  By construction, the composite $\mathcal{P}_{\mathrm{frame}} \bullet \mathcal{P} \bullet \mathcal{P}_n$ is defined and $\mathcal{P}_{\mathrm{frame}} \bullet \mathcal{P} \bullet \mathcal{P}_n \sqsubseteq (\Sigma_{\mathrm{cl}}, \mu')$ and X is $\mathcal{P}_n$-measurable and uniformly distributed in $[0,1]$. Therefore $\gamma, (D, X), \mathcal{P} \bullet \mathcal{P}_n \Vdash Q$ by (1), and it only remains to show

$$\begin{pmatrix} \omega \leftarrow \mu; \\ v \leftarrow \mathrm{Unif}\ [0,1]; \\ \mathrm{ret}\ (D_{\mathrm{est}}(\omega), D(\omega), v) \end{pmatrix} = \begin{pmatrix} \omega \leftarrow \mu'; \\ \mathrm{ret}\ (D_{\mathrm{est}}(\omega), D(\omega), X(\omega)) \end{pmatrix}$$

$$\begin{pmatrix} \omega \leftarrow \mu; \\ v \leftarrow \mathrm{Unif}\ [0,1]; \\ \mathrm{ret}\ (D_{\mathrm{est}}(\omega), D(\omega), v) \end{pmatrix} = \begin{pmatrix} \omega_{:n} \leftarrow \mu|_{\{0,1\}^n}; \\ v \leftarrow \mathrm{Unif}\ [0,1]; \\ \mathrm{ret}\ (D_{\mathrm{est}}(\omega_{:n}, \_), D(\omega_{:n}, \_), v) \end{pmatrix}$$

$$= \begin{pmatrix} (\omega_{:n}, \omega, v) \leftarrow \mu|_{\{0,1\}^n} \otimes \mathrm{Unif}\ [0,1]; \\ \mathrm{ret}\ (D_{\mathrm{est}}(\omega_{:n}, \_), D(\omega_{:n}, \_), v) \end{pmatrix} = \begin{pmatrix} \omega_{:n} \leftarrow \mu|_{\{0,1\}^n}; \\ \mathrm{ret}\ (D_{\mathrm{est}}(\omega_{:n}, \_), D(\omega_{:n}, \omega_{n+1}) \end{pmatrix}$$

fp wit meas pcm bind post